

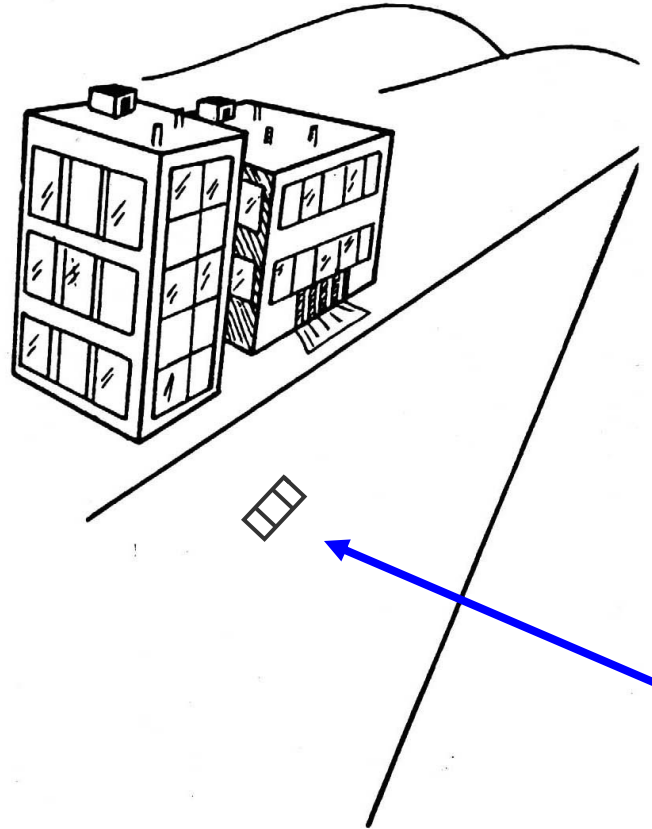
Image Segmentation

How many zebras?



From [Sandlot Science](#)

Why context is important?

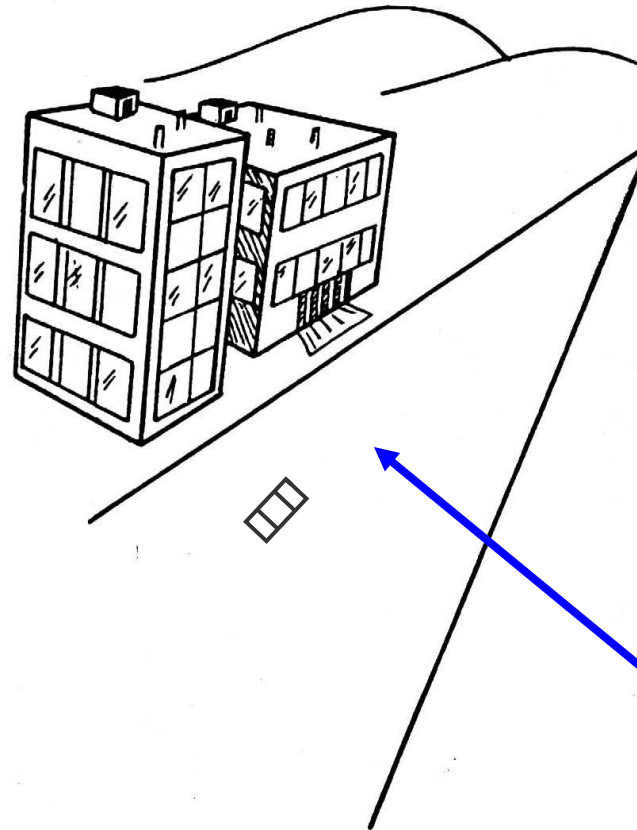


What is this?

Why is this a car?

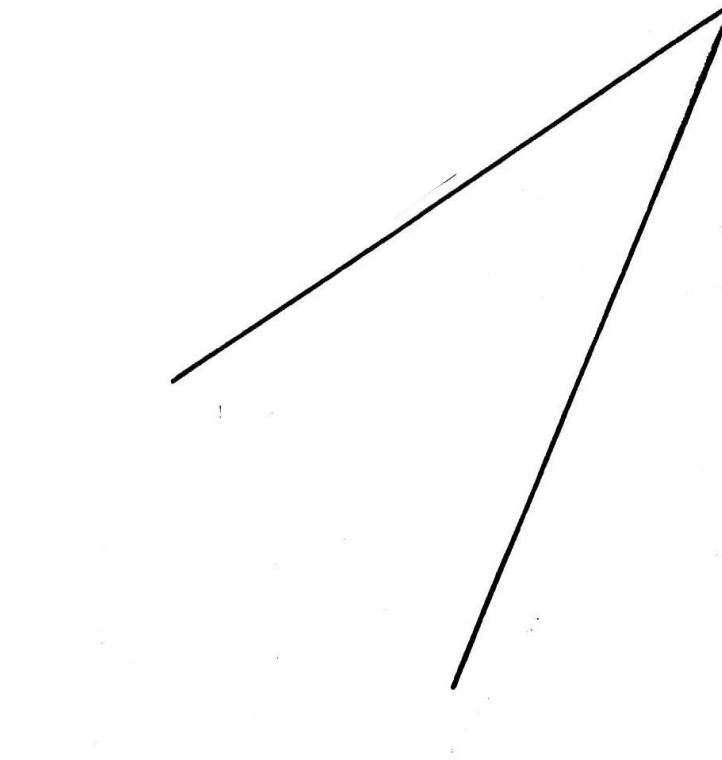


...because it's on the road!



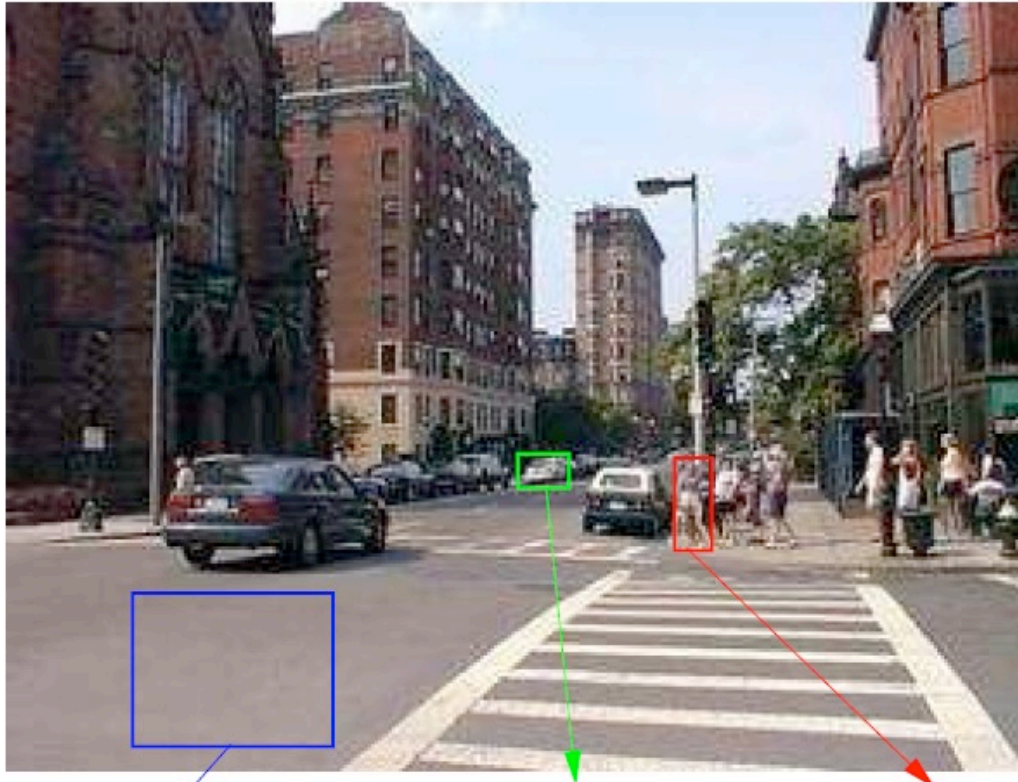
Why is this road?

Why is this a road?

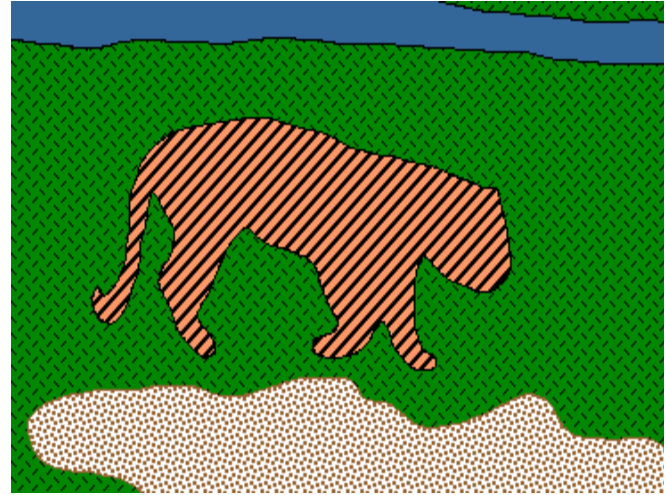


Context is very important !

Same problem in real scenes



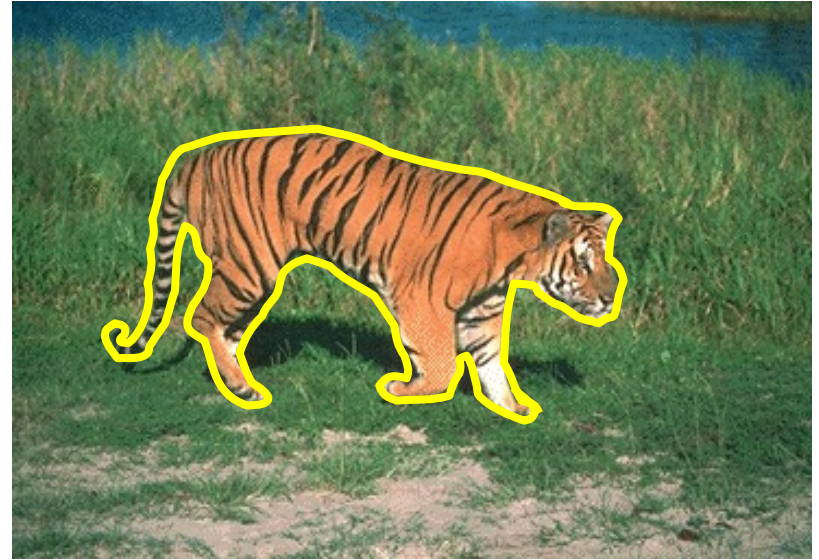
From images to objects



What defines an object?

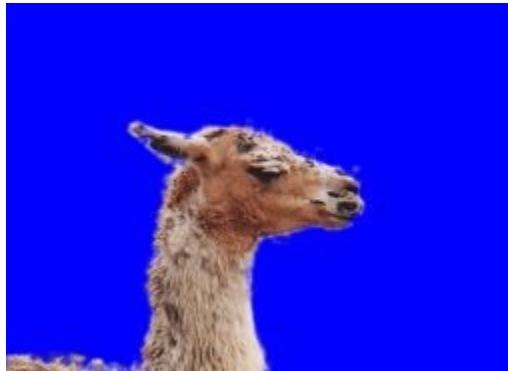
- Subjective problem, but has been well-studied

Extracting objects



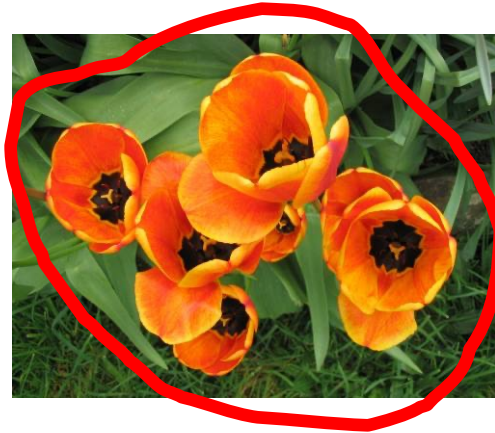
How could we do this automatically (or at least semi-automatically)?

Semi-automatic binary segmentation

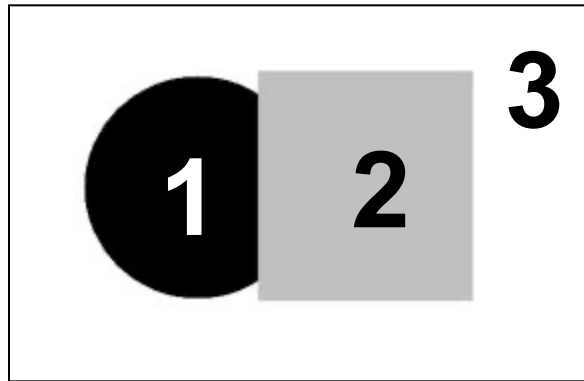


Simplifying the user interaction

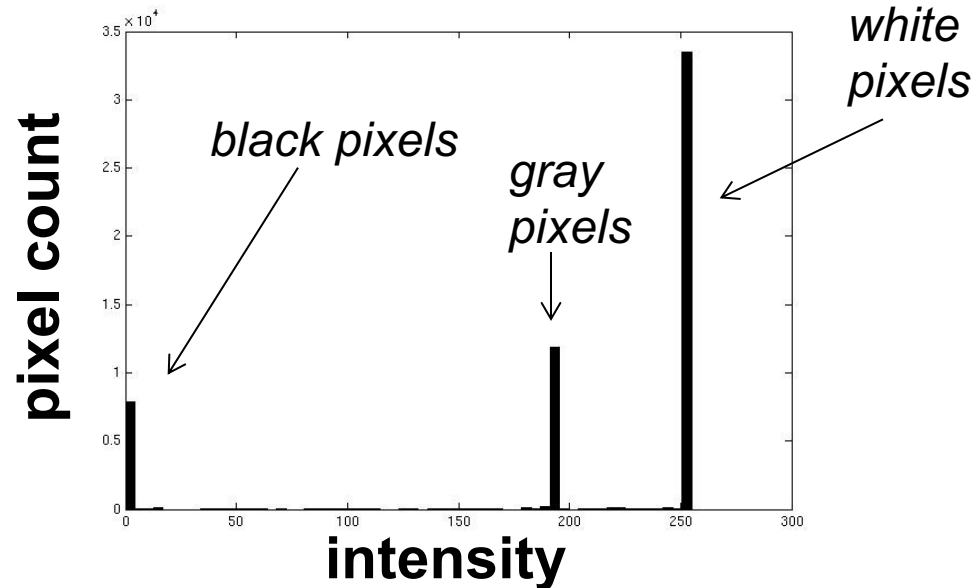
Grabcut [[Rother et al., SIGGRAPH 2004](#)]



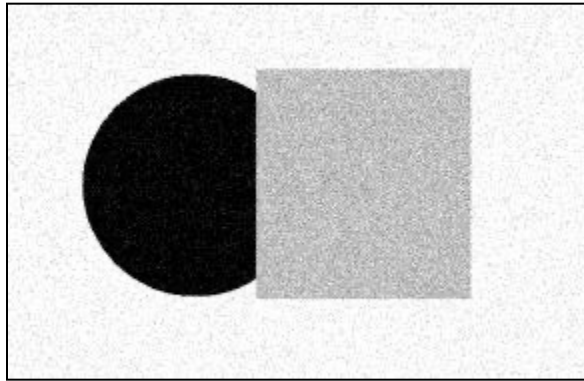
Auto segmentation: toy example



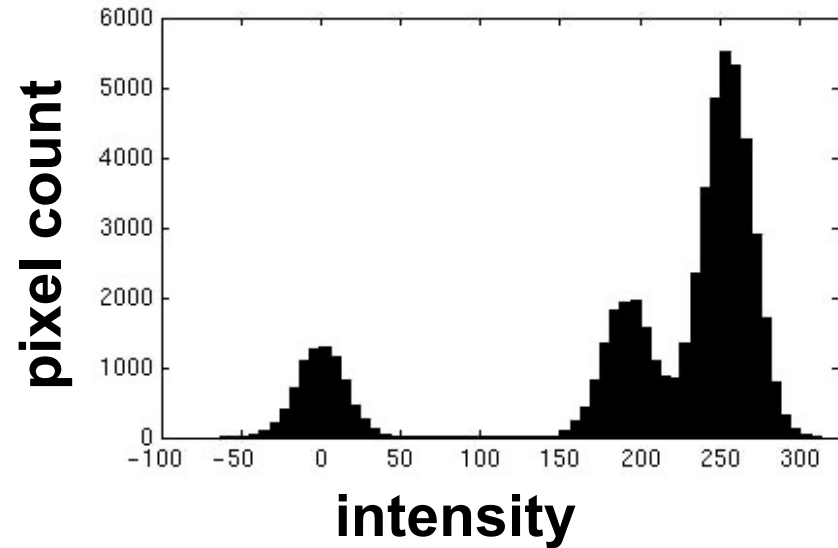
input image



- These intensities define the three groups.
- We could label every pixel in the image according to which of these primary intensities it is.
 - i.e., *segment* the image based on the intensity feature.
- But ... image isn't quite so simple ...



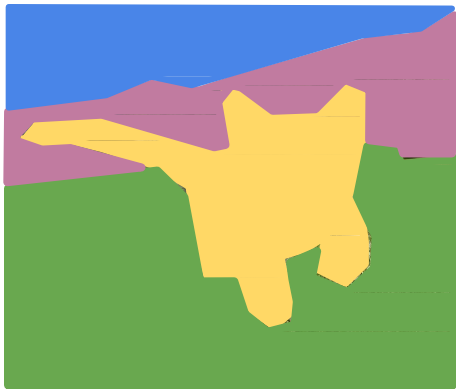
input image



- Now how to determine the three main intensities that define our groups?
- We need to ***cluster***.

Deep Learning

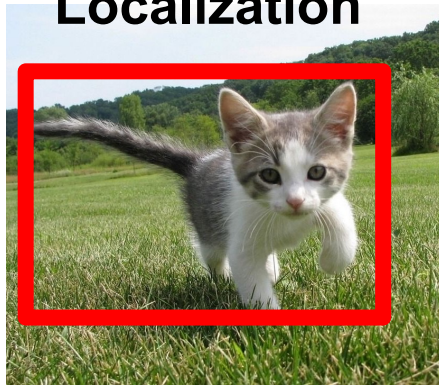
Semantic Segmentation



GRASS, CAT,
TREE, SKY

Pixel-level

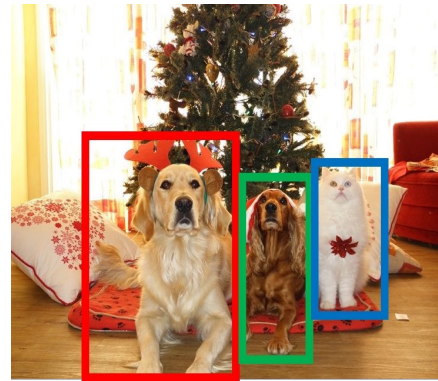
**Classification
+
Localization**



CAT

Single Object

**Object
Detection**



DOG, DOG, CAT

Multiple Object

**Instance
Segmentation**

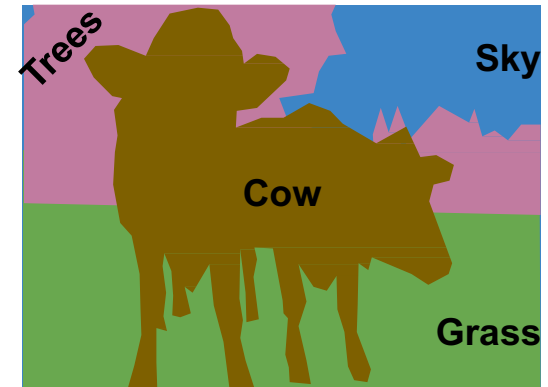
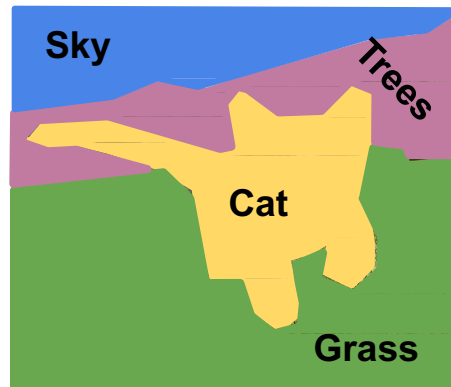


DOG, DOG, CAT

Segmentation+Classification

Semantic Segmentation

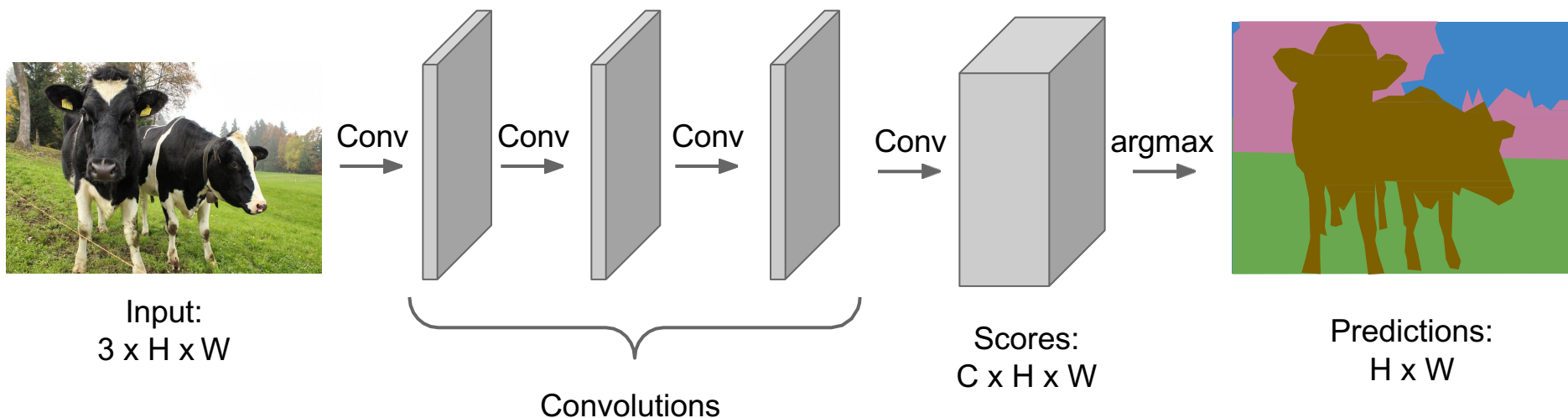
Label each pixel in the image with a category label



Don't differentiate instances, only care about pixels

Semantic Segmentation Idea: Fully Convolutional

Design a network as a bunch of convolutional layers
to make predictions for pixels all at once!



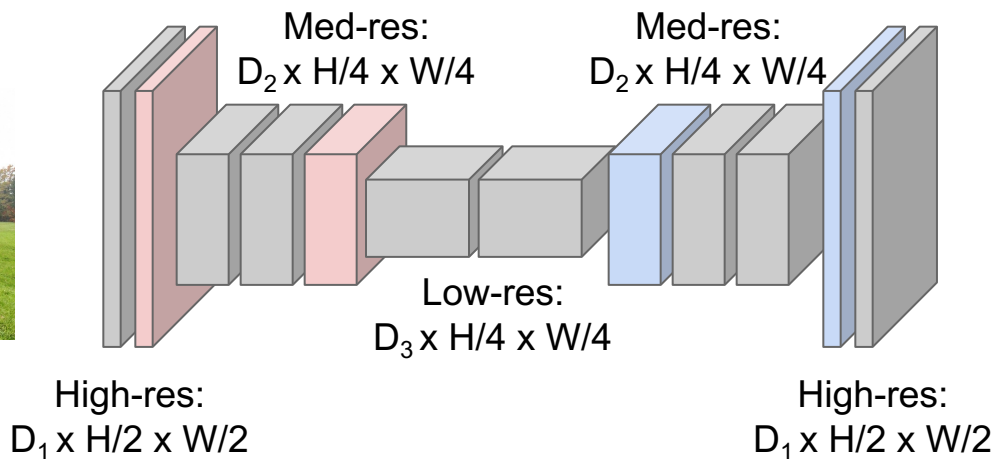
Each channel is a class
 C channels $\rightarrow$ C classes

Semantic Segmentation Idea: Fully Convolutional

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!



Input:
 $3 \times H \times W$



Predictions:
 $H \times W$

Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015
Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

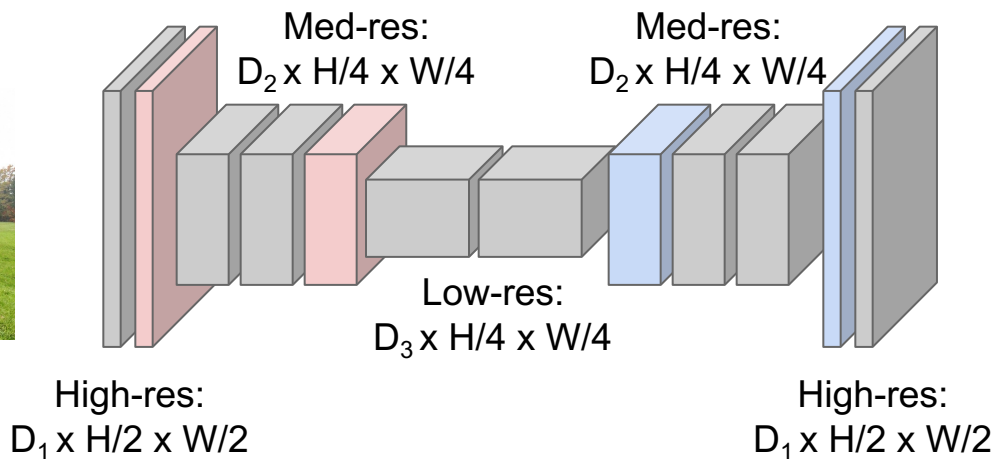
Semantic Segmentation Idea: Fully Convolutional

Downsampling:
Pooling, strided
convolution



Input:
 $3 \times H \times W$

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!



Upsampling:
???



Predictions:
 $H \times W$

Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015
Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

In-Network upsampling: “Unpooling”

Nearest Neighbor

1	2
3	4



1	1	2	2
1	1	2	2
3	3	4	4
3	3	4	4

Input: 2 x 2

Output: 4 x 4

“Bed of Nails”

1	2
3	4

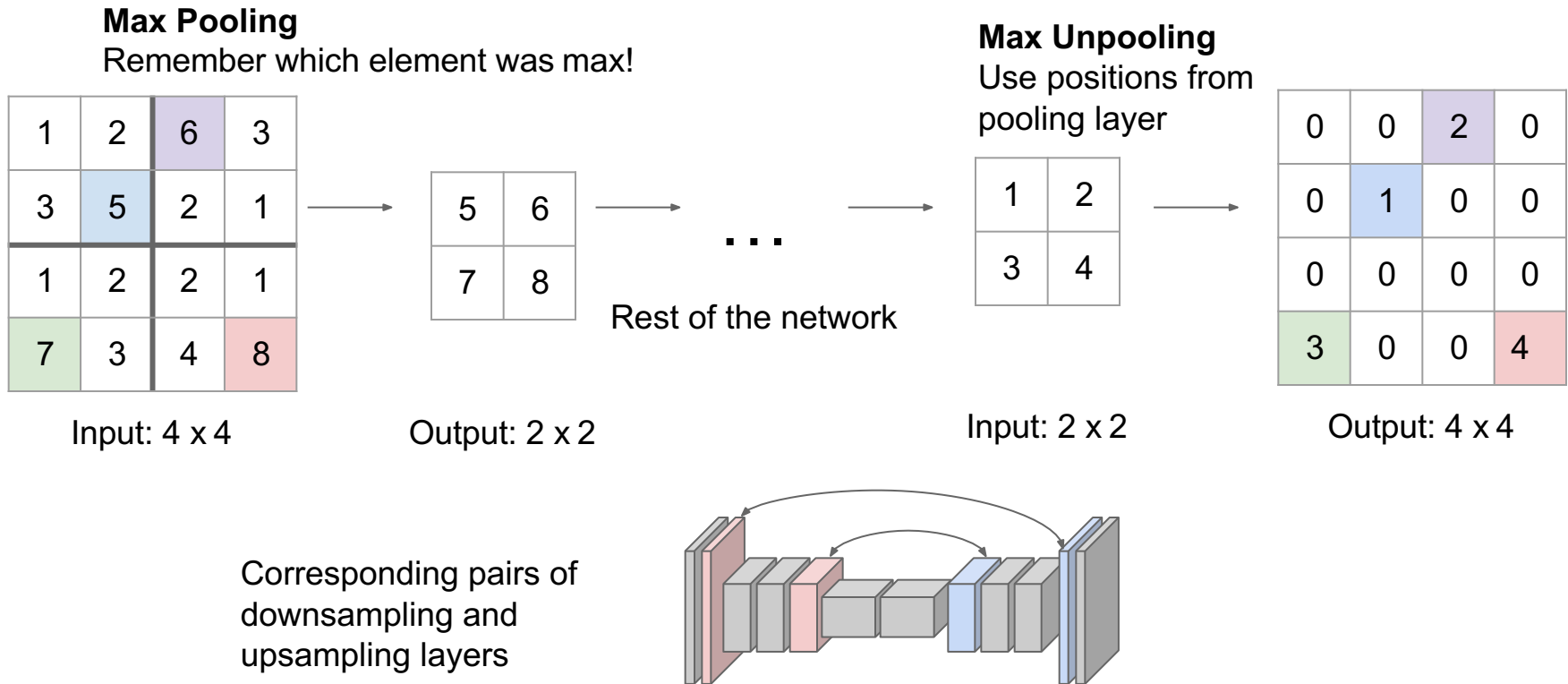


1	0	2	0
0	0	0	0
3	0	4	0
0	0	0	0

Input: 2 x 2

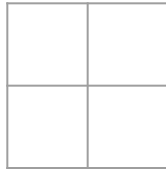
Output: 4 x 4

In-Network upsampling: “Max Unpooling”

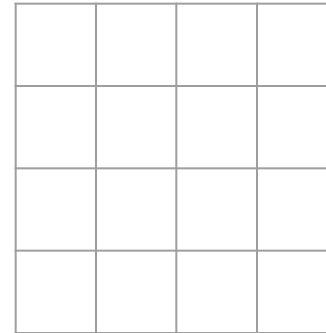


Learnable Upsampling

3 x 3 **transpose** convolution, stride 2 pad 1



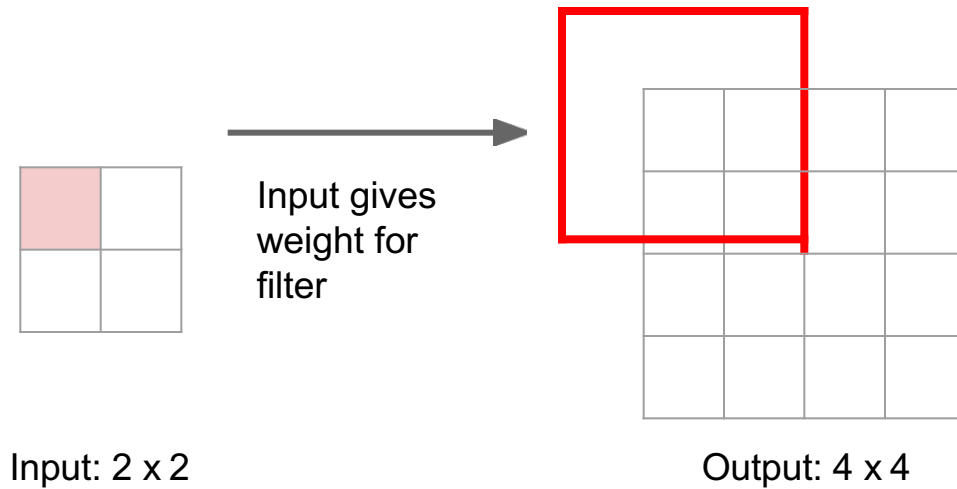
Input: 2 x 2



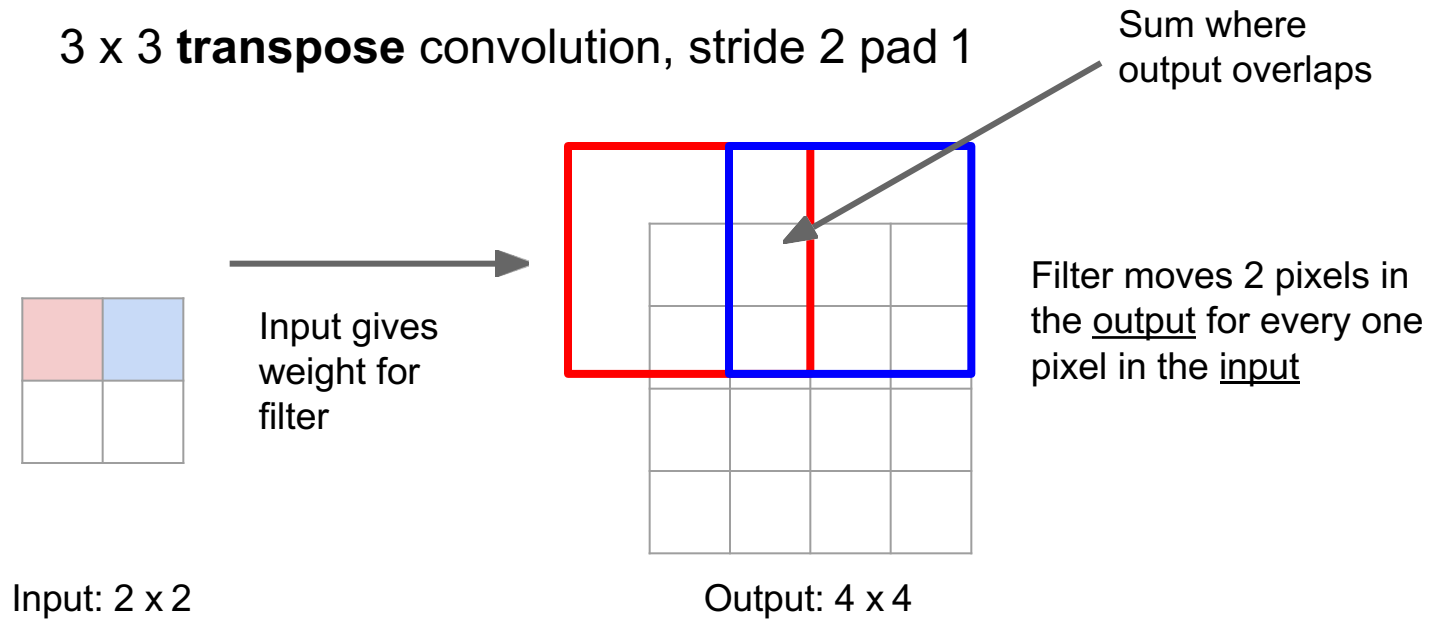
Output: 4 x 4

Learnable Upsampling

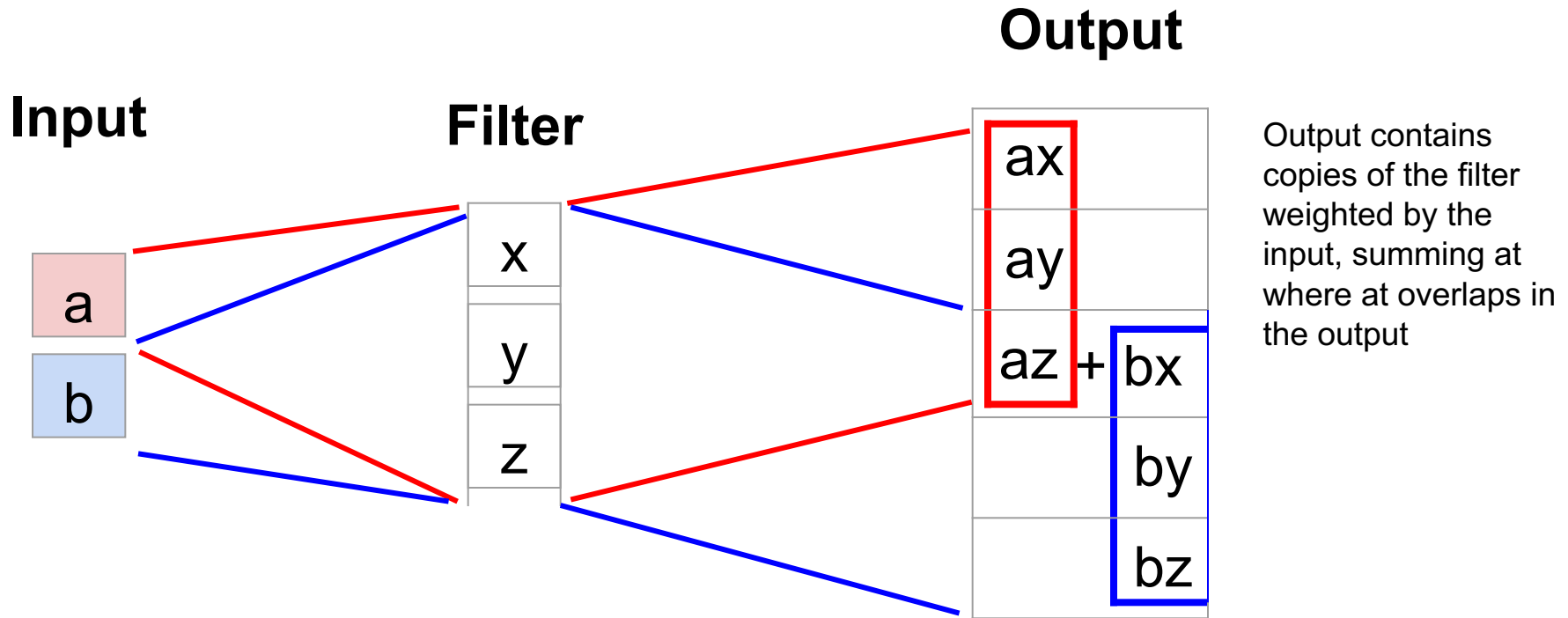
3 x 3 **transpose** convolution, stride 2 pad 1



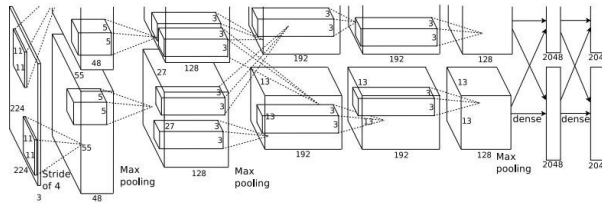
Learnable Upsampling



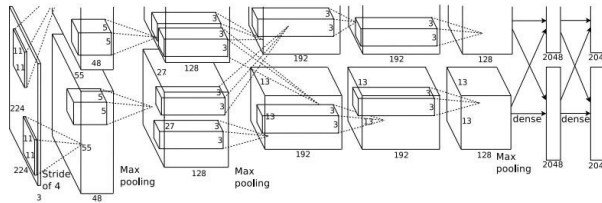
Transpose Convolution: 1D Example



Object Detection as Regression?



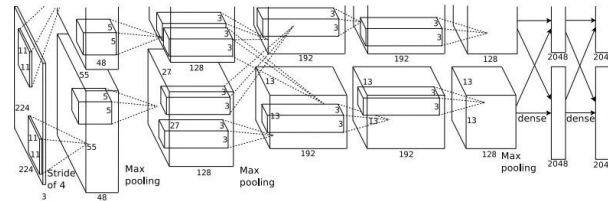
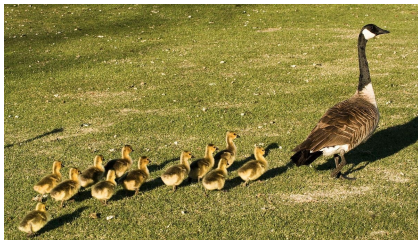
CAT: (x, y, w, h)



DOG: (x, y, w, h)

DOG: (x, y, w, h)

CAT: (x, y, w, h)

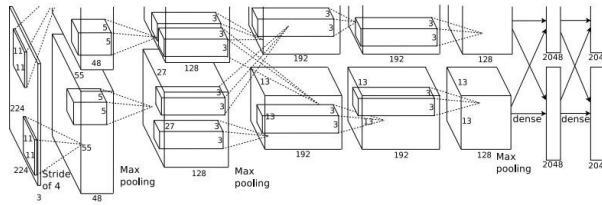


DUCK: (x, y, w, h)

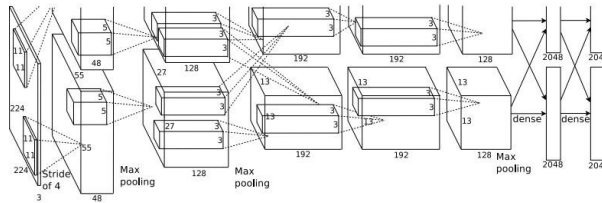
DUCK: (x, y, w, h)

....

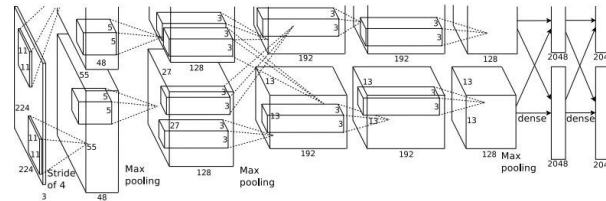
Object Detection as Regression?



CAT: (x, y, w, h) 4 numbers



DOG: (x, y, w, h)
DOG: (x, y, w, h) 16 numbers
CAT: (x, y, w, h)



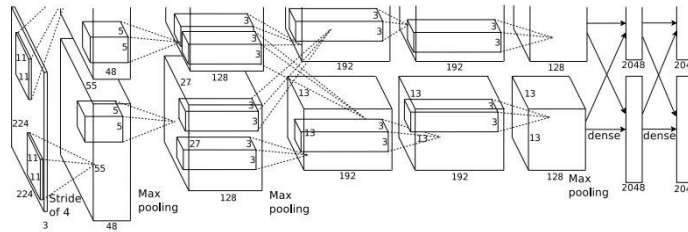
DUCK: (x, y, w, h) Many
DUCK: (x, y, w, h) numbers!

....

Each image needs a different
number of outputs!

Object Detection as Classification: Sliding Window

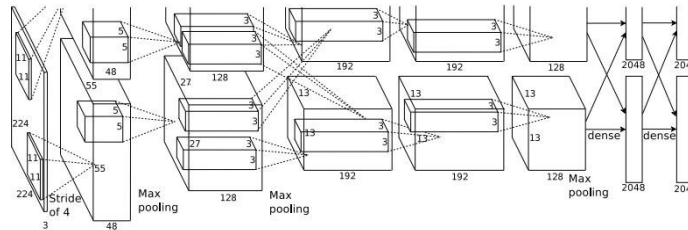
Apply a CNN to many different crops of the image, CNN classifies each crop as object or background



Dog? NO Cat?
NO
Background? YES

Object Detection as Classification: Sliding Window

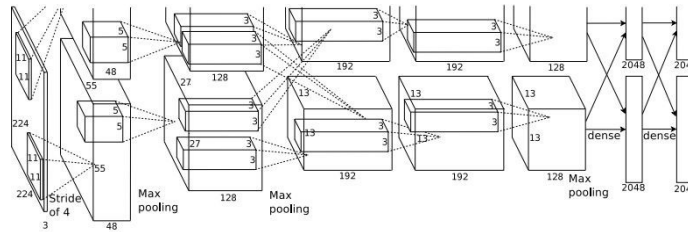
Apply a CNN to many different crops of the image, CNN classifies each crop as object or background



Dog? YES
Cat? NO
Background? NO

Object Detection as Classification: Sliding Window

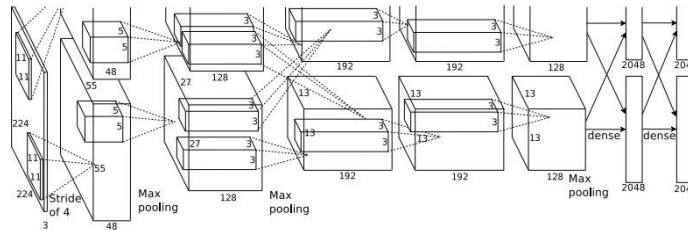
Apply a CNN to many different crops of the image, CNN classifies each crop as object or background



Dog? YES
Cat? NO
Background? NO

Object Detection as Classification: Sliding Window

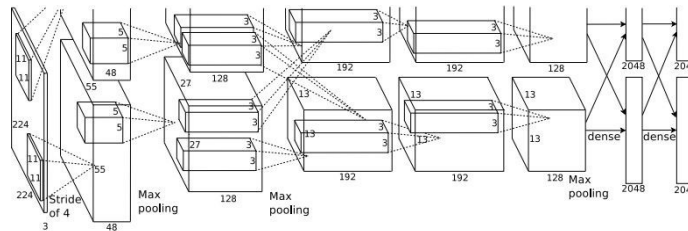
Apply a CNN to many different crops of the image, CNN classifies each crop as object or background



Dog? NO Cat?
YES
Background? NO

Object Detection as Classification: Sliding Window

Apply a CNN to many different crops of the image, CNN classifies each crop as object or background

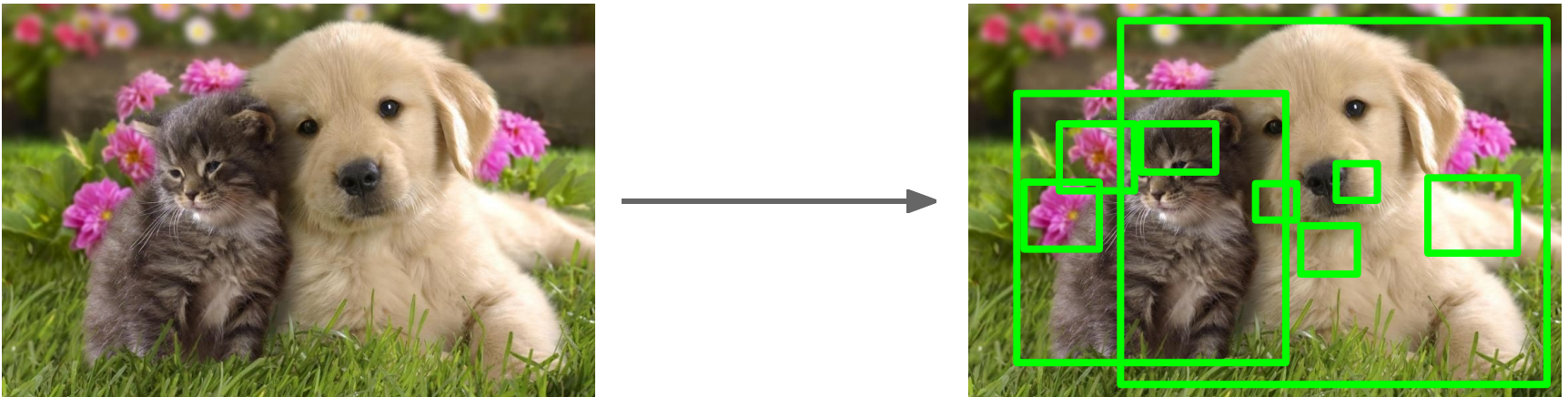


Dog? NO Cat? YES
Background? NO

Problem: Need to apply CNN to huge number of locations and scales, very computationally expensive!

Region Proposals

- Find image regions that are likely to contain objects
- Relatively fast to run; e.g. Selective Search gives 1000 region proposals in a few seconds on CPU

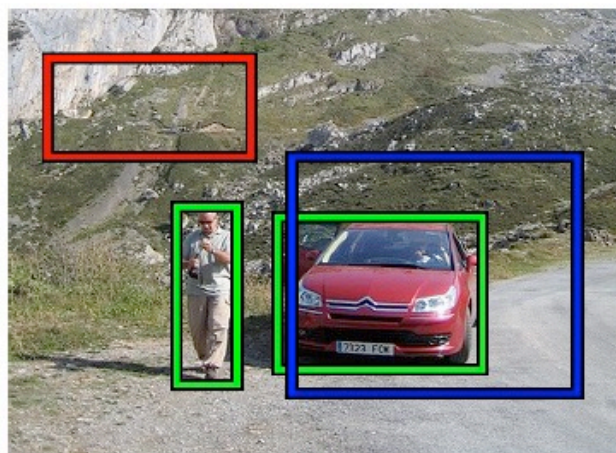


Alexe et al, “Measuring the objectness of image windows”, TPAMI 2012

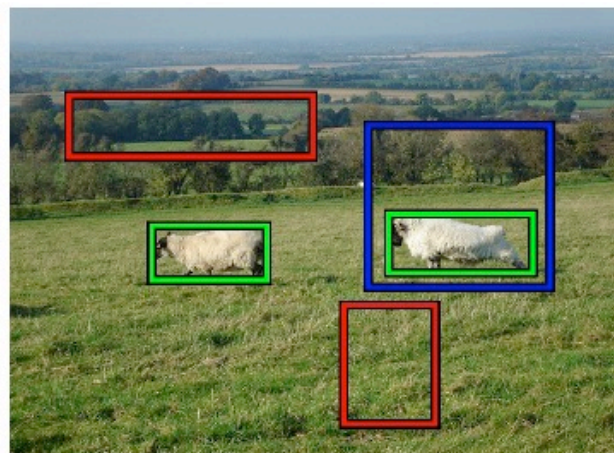
Uijlings et al, “Selective Search for Object Recognition”, IJCV 2013

Cheng et al, “BING: Binarized normed gradients for objectness estimation at 300fps”, CVPR 2014

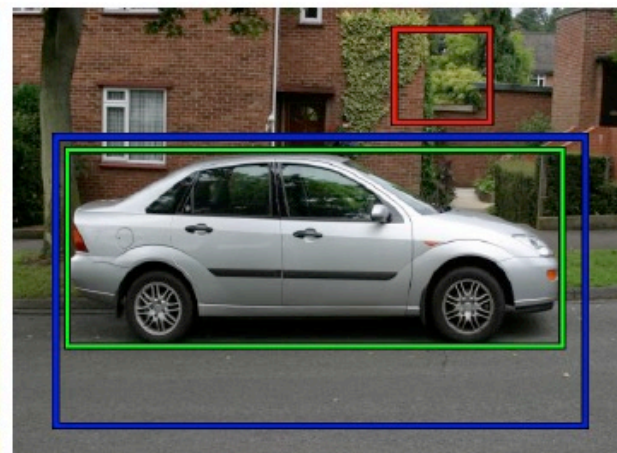
Zitnick and Dollar, “Edge boxes: Locating object proposals from edges”, ECCV 2014



(a)



(b)



(c)

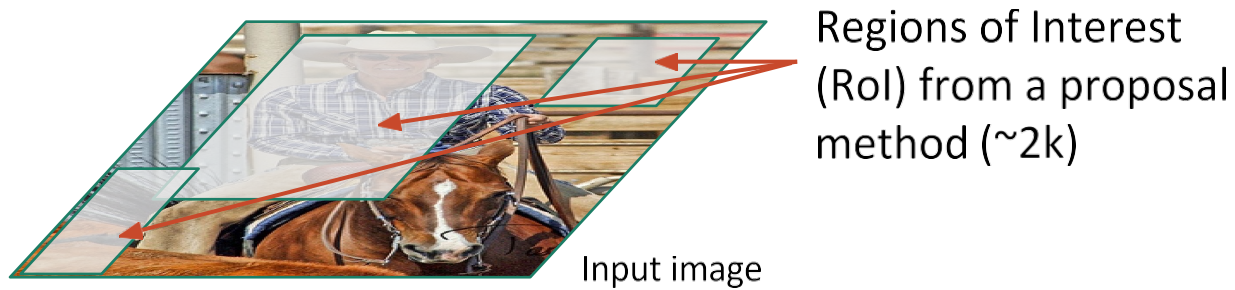
Fig. 1: Desired behavior of an objectness measure. *The desired objectness measure should score the blue windows, partially covering the objects, lower than the ground truth windows (green), and score even lower the red windows containing only stuff or small parts of objects.*

R-CNN

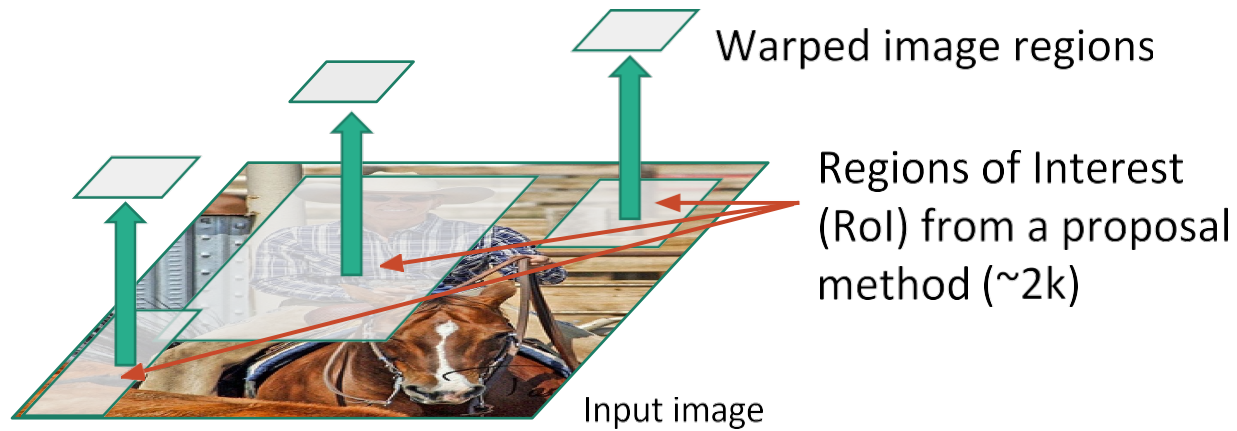


Input image

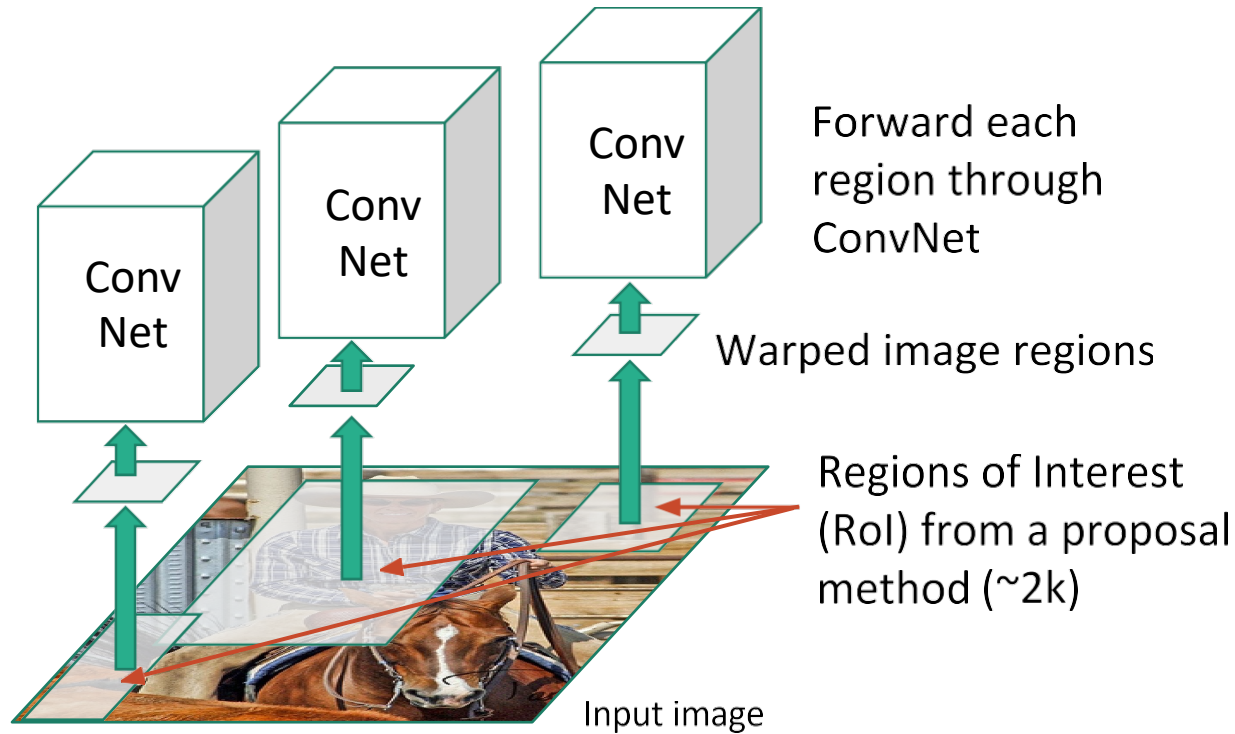
R-CNN



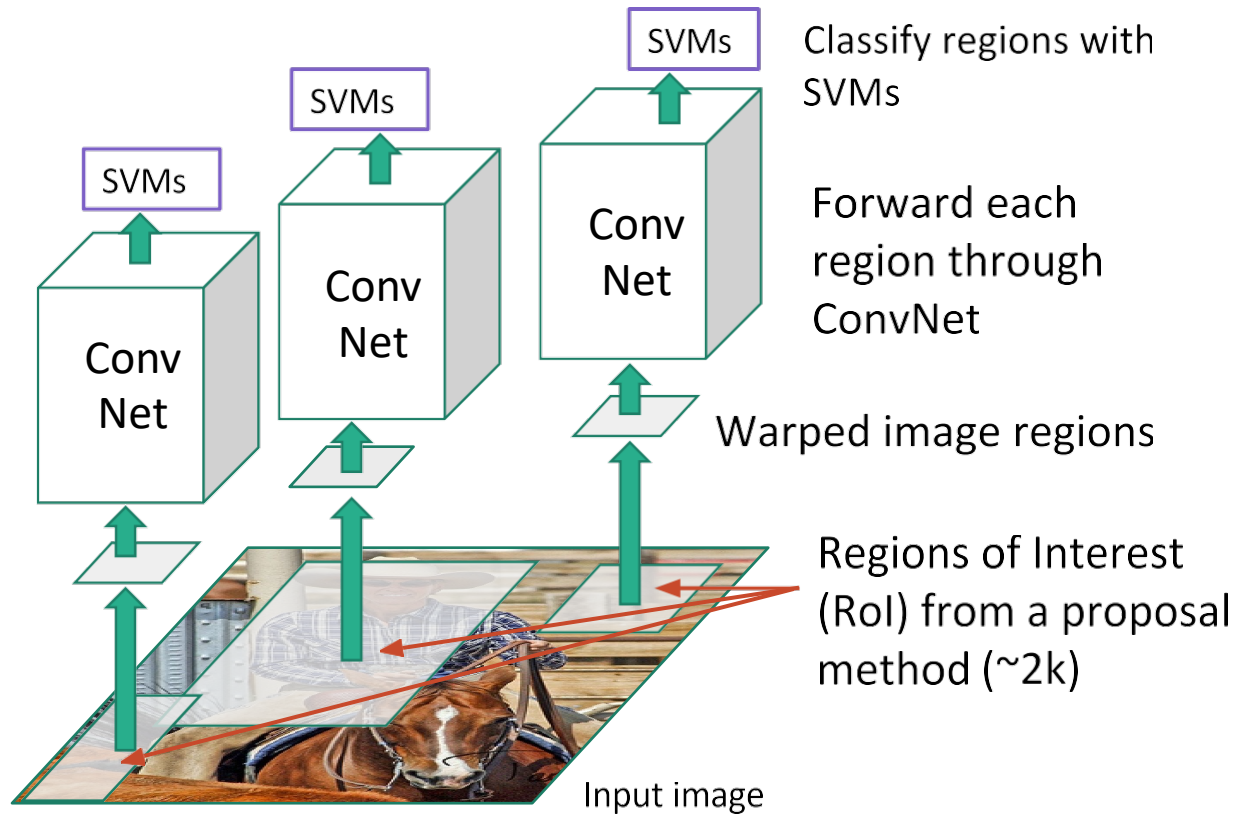
R-CNN



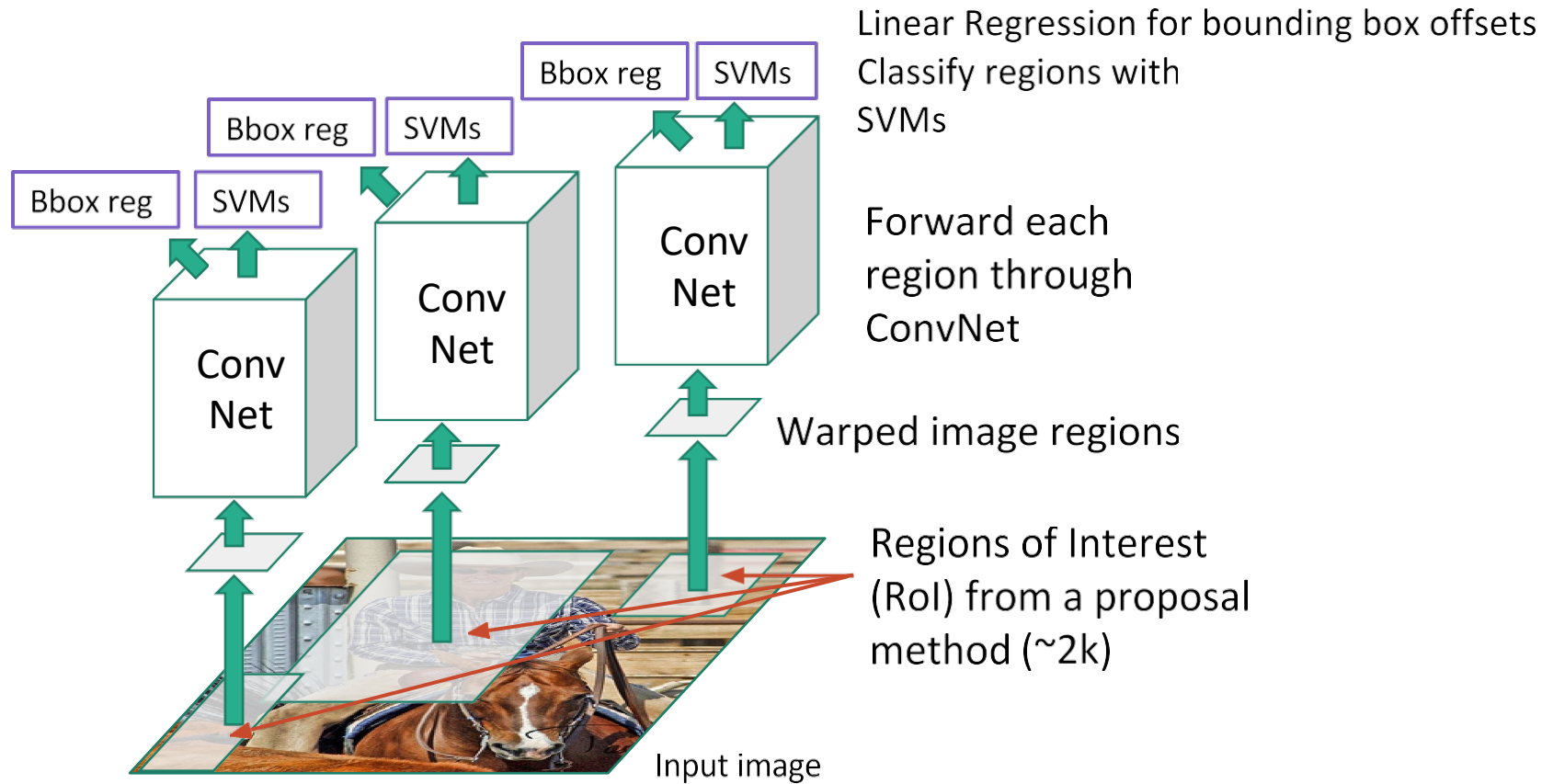
R-CNN



R-CNN



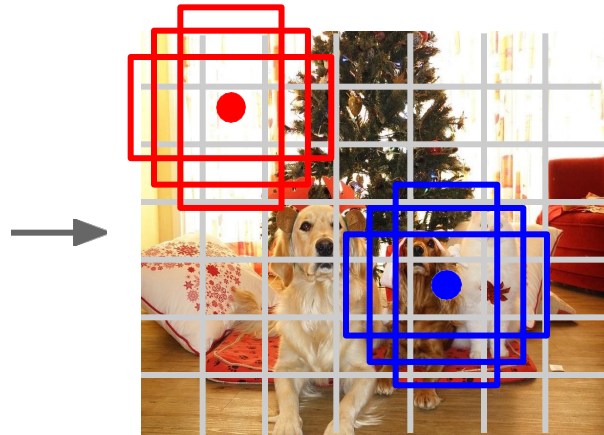
R-CNN



Detection without Proposals: YOLO



Input image
 $3 \times H \times W$



Divide image into grid 7×7

Image a set of **base boxes** centered at each grid cell Here $B = 3$

Within each grid cell:

- Regress from each of the B base boxes to a final box with 5 numbers: $(dx, dy, dh, dw, \text{confidence})$
- Predict scores for each of C classes (including background as a class)

Output:
 $7 \times 7 \times (5 * B + C)$

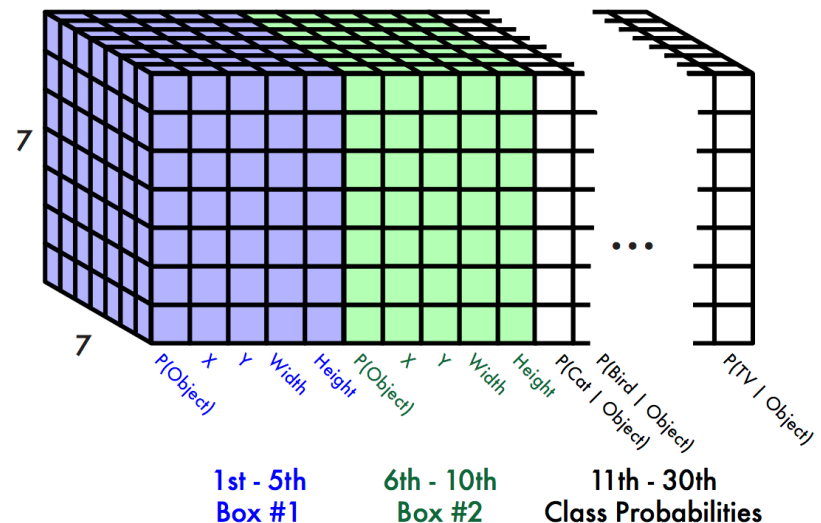
This parameterization fixes the output size

- Each cell predicts:
 - For each bounding box:
 - 4 coordinates (x, y, w, h)
 - 1 confidence value
 - Some number of class probabilities

- For Pascal VOC:

- 7x7 grid
- 2 bounding boxes / cell
- 20 classes

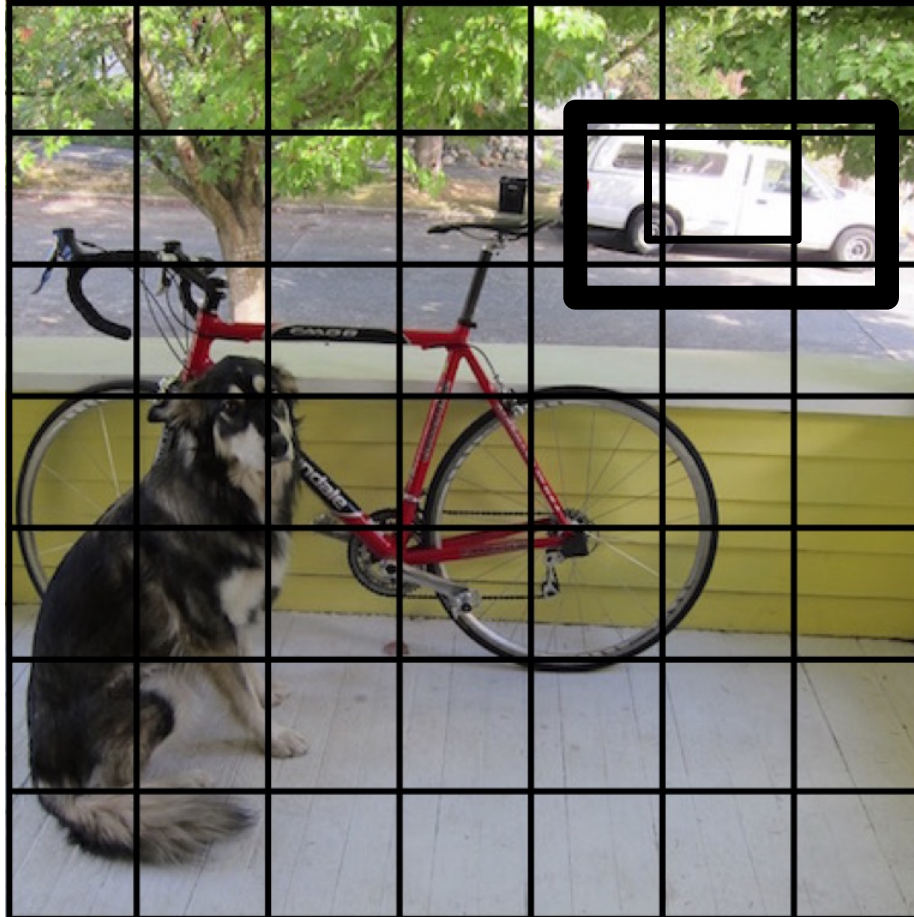
- $7 \times 7 \times (2 \times 5 + 20) = 7 \times 7 \times 30$ tensor = **1470 outputs**



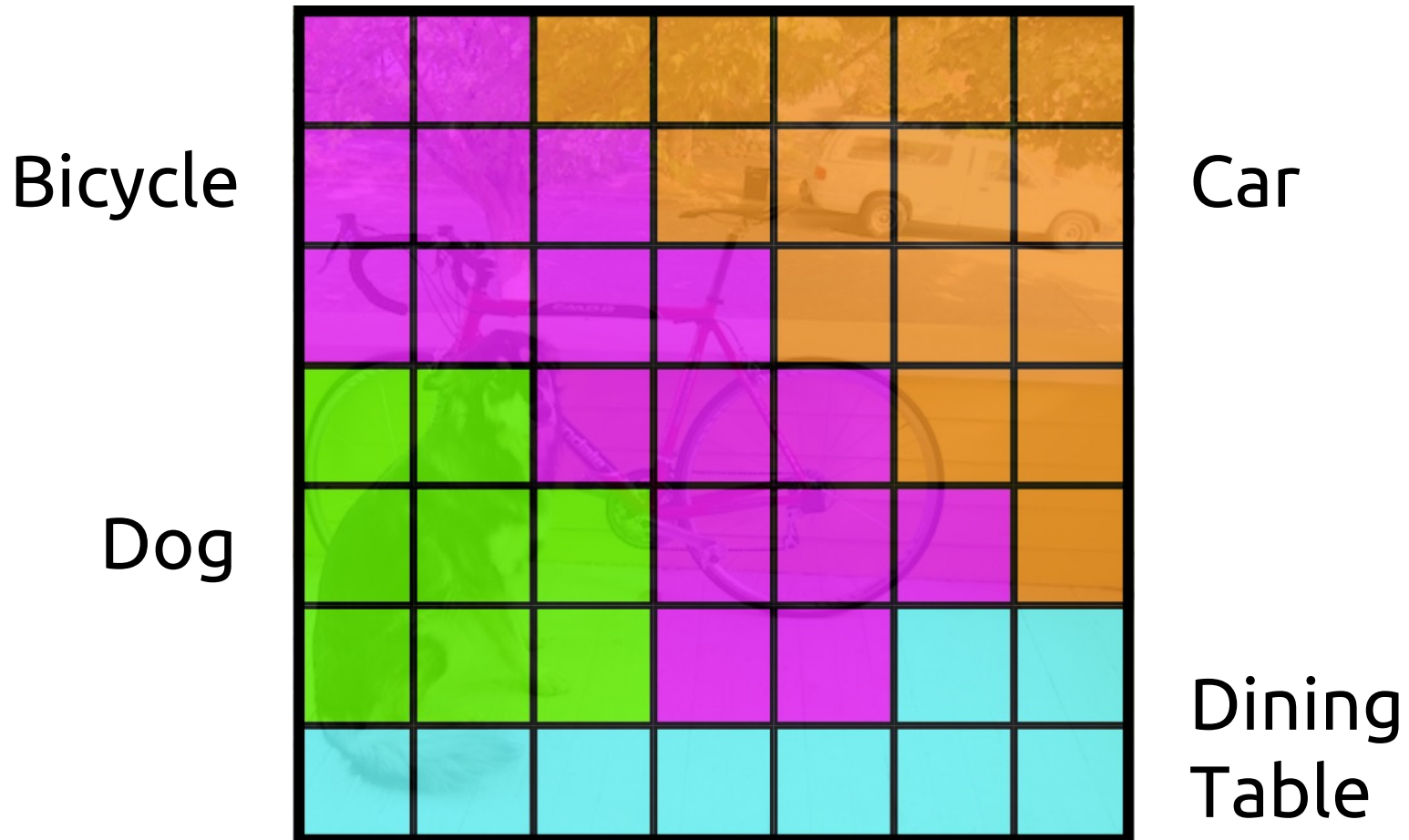
Split the image into a grid



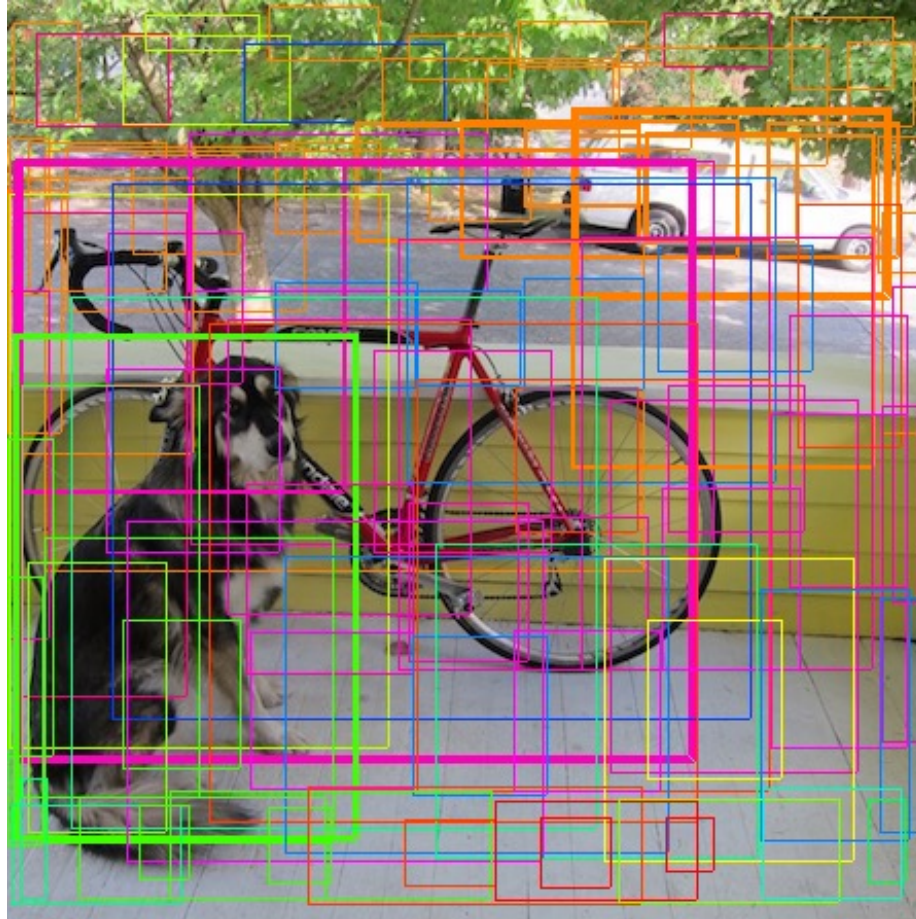
Each cell predicts boxes and confidences: $P(\text{Object})$



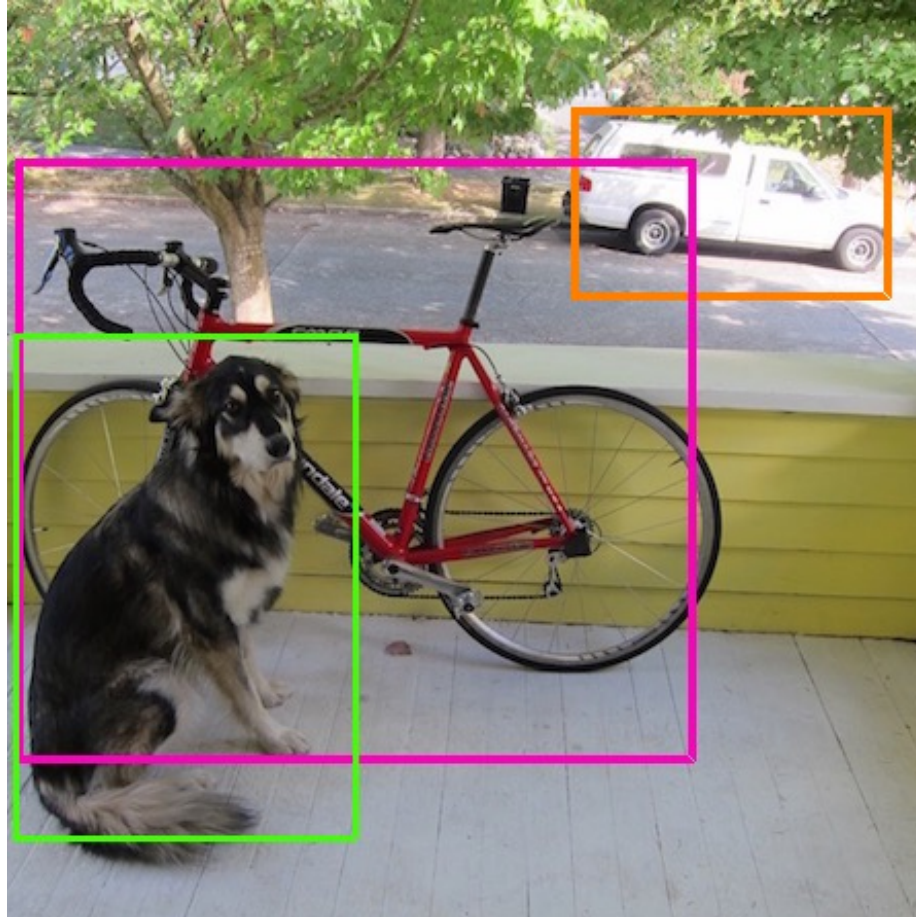
Each cell also predicts a probability
 $P(\text{Class} \mid \text{Object})$



Combine the box and class predictions



Finally do non-maximum suppression and threshold detections



It also generalizes well to new domains

